# The Dispatcher: Stage Three of a Verified Reader, Movement Three

Working paper v0.1, Project secret3, August 11, 2026


## Abstract

Movement three composes the two eyes into one organ. The dispatcher rootRead applies the K eye to its quoted input and cases the tagged answer: a just passes through, rewrapped by the internal just constructor jWrap; a nothing hands the original quote to the S eye. Nine theorems then cover the code space exactly: rootReadK and rootReadS fire, delivering the kept child and the assembled contractum respectively, and seven silence theorems answer nothing wherever the root is no redex. Every proof is a four-link composition, the eyes' theorems lifted under two left congruences and closed by a tag case; no new bench machinery was needed, as movement two predicted. With this, everything below the knot is standing: the alphabet, the engine as specification, the bench through arity four, the tags, the constructor, both eyes, and the root dispatcher, all composition-verified. The paper closes by stating movement four's obligations precisely, as the campaign's remaining specification.


## 1. The organ

The composition uses the tags as control: rootRead is one abstraction whose body applies the K eye's answer, a Scott result, to two continuations, the S eye's pending application as the nothing branch and the just constructor as the just branch. The just constructor jWrap is itself a one-line internal term with a one-line beta theorem. Nothing else was built. The nine dispatch theorems are, deliberately, boring: each is the root beta, the relevant K-eye theorem lifted by two left congruences into the application spine, a single tag case, and, on the nothing path, the relevant S-eye theorem verbatim. The longest proof in the movement is four links. This is the payoff of the campaign's economy: by movement three, correctness proofs are catalog work.

## 2. Exact agreement with the engine's root fragment

The engine simStep does two things: it fires root redexes and it descends. The dispatcher is the compiled internalization of the first: for every code, rootRead answers just the root contractum exactly when the engine's root clauses would rewrite, and nothing exactly when they would not, the nine theorems partitioning the code space as the eyes' theorems did before them, now through one interface. The descent is deliberately absent, because descent is where self-reference begins, and self-reference is movement four.

## 3. The campaign ledger

Standing and composition-verified after three movements: the quoted alphabet with closure and injectivity; the engine as reference semantics; the bench, sub1NoOcc through abs4Beta, every side condition discharged by quoteClosed; the internal tags with their cases; the constructor qMk; the K eye, seven theorems; the S eye with the assembler, nine theorems; and the dispatcher, nine more. Twenty five behavior theorems, three consecutive first-attempt compilations, one lemma catalog.

### 4. Movement four, specified

What remains is the knot, and its obligations can now be written down exactly. First, the recursor's body: a step function F of two abstractions, self and quote, whose root branch is the dispatcher above, and whose descent branch must case the quote's application form, recurse self on the left child, on a just rebuild with qMk against the untouched right child, on a nothing recurse self on the right child, and rebuild symmetrically; every piece of this body except self is already on the bench. Second, the knot itself: Sim is the fixed point of F through the module's yKnot, so that App Sim q unfolds, by the compiled ladder, to App (App F Sim) q. Third, correctness: for every code c, if simStep c is just c' then App Sim (quote c) walks to jTag (quote c'), and if nothing, to nTag, proved by induction on c with the unfolding lemma interleaved into every chain, the congruence-lifted composition rehearsed in movement two now carrying recursive calls in argument position. Nothing in the specification is mysterious any longer; what remains is the discipline of executing it, and the campaign's record suggests the catalog will hold.

### Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 3,011 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification.

### Appendix B: The dispatcher section (module part W)

```
--------------------------------------------------------------------
-- Tier 2, part W: the dispatcher (C5 stage three, movement three)
--------------------------------------------------------------------

||| the just constructor as a term: wrap a payload into its tag
jWrap : Term
jWrap = abs "v"
  (App CK (App (App CS Proofs.iC) (App CK (Var "v"))))

jWrapBeta : (w : Term) ->
  Steps (App Proofs.jWrap w) (Proofs.jTag w)
jWrapBeta w =
  absCorrect "v"
    (App CK (App (App CS Proofs.iC) (App CK (Var "v")))) w

||| the dispatcher: try the K eye; on nothing, the S eye
rootRead : Term
rootRead = abs "q1"
  (App (App (App Proofs.readK (Var "q1"))
            (App Proofs.readS (Var "q1")))
       Proofs.jWrap)

rootBeta : (c : Code) ->
  Steps (App Proofs.rootRead (Proofs.quote c))
        (App (App (App Proofs.readK (Proofs.quote c))
                  (App Proofs.readS (Proofs.quote c)))
             Proofs.jWrap)
rootBeta c =
  absCorrect "q1"
    (App (App (App Proofs.readK (Var "q1"))
              (App Proofs.readS (Var "q1")))
         Proofs.jWrap)
    (quote c)
```

```
||| the dispatcher fires on a root K redex
rootReadK : (a, y : Code) ->
  Steps (App Proofs.rootRead (Proofs.quote (QApp (QApp QK a) y)))
        (Proofs.jTag (Proofs.quote a))
rootReadK a y =
  stepsTrans (rootBeta (QApp (QApp QK a) y)) $
  stepsTrans (appLs (appLs (readKFires a y))) $
  stepsTrans (jTagCase (quote a)
    (App readS (quote (QApp (QApp QK a) y))) jWrap) $
  jWrapBeta (quote a)

||| the dispatcher fires on a root S redex and hands over the
||| assembled contractum
rootReadS : (a, b, g : Code) ->
  Steps (App Proofs.rootRead
          (Proofs.quote (QApp (QApp (QApp QS a) b) g)))
        (Proofs.jTag (Proofs.quote (QApp (QApp a g) (QApp b g))))
rootReadS a b g =
  stepsTrans (rootBeta (QApp (QApp (QApp QS a) b) g)) $
  stepsTrans (appLs (appLs (readKDeep QS a b g))) $
  stepsTrans (nTagCase
    (App readS (quote (QApp (QApp (QApp QS a) b) g))) jWrap) $
  readSFires a b g

||| and stays silent wherever the root is no redex
rootReadAtomS : Steps (App Proofs.rootRead (Proofs.quote QS)) Proofs.nTag
rootReadAtomS =
  stepsTrans (rootBeta QS) $
  stepsTrans (appLs (appLs readKAtomS)) $
  stepsTrans (nTagCase (App readS (quote QS)) jWrap) $
  readSAtomS

rootReadAtomK : Steps (App Proofs.rootRead (Proofs.quote QK)) Proofs.nTag
rootReadAtomK =
  stepsTrans (rootBeta QK) $
  stepsTrans (appLs (appLs readKAtomK)) $
  stepsTrans (nTagCase (App readS (quote QK)) jWrap) $
  readSAtomK

rootReadL1S : (y : Code) ->
  Steps (App Proofs.rootRead (Proofs.quote (QApp QS y))) Proofs.nTag
rootReadL1S y =
  stepsTrans (rootBeta (QApp QS y)) $
  stepsTrans (appLs (appLs (readKAppS y))) $
  stepsTrans (nTagCase (App readS (quote (QApp QS y))) jWrap) $
  readSL1S y

rootReadL1K : (y : Code) ->
  Steps (App Proofs.rootRead (Proofs.quote (QApp QK y))) Proofs.nTag
rootReadL1K y =
  stepsTrans (rootBeta (QApp QK y)) $
  stepsTrans (appLs (appLs (readKAppK y))) $
  stepsTrans (nTagCase (App readS (quote (QApp QK y))) jWrap) $
  readSL1K y
```

```
rootReadL2S : (a, y : Code) ->
  Steps (App Proofs.rootRead (Proofs.quote (QApp (QApp QS a) y)))
        Proofs.nTag
rootReadL2S a y =
  stepsTrans (rootBeta (QApp (QApp QS a) y)) $
  stepsTrans (appLs (appLs (readKAppSS a y))) $
  stepsTrans (nTagCase (App readS (quote (QApp (QApp QS a) y))) jWrap) $
  readSL2S a y

rootReadL3K : (a, b, y : Code) ->
  Steps (App Proofs.rootRead
          (Proofs.quote (QApp (QApp (QApp QK a) b) y)))
        Proofs.nTag
rootReadL3K a b y =
  stepsTrans (rootBeta (QApp (QApp (QApp QK a) b) y)) $
  stepsTrans (appLs (appLs (readKDeep QK a b y))) $
  stepsTrans (nTagCase
    (App readS (quote (QApp (QApp (QApp QK a) b) y))) jWrap) $
  readSL3K a b y

rootReadL3Deep : (p, q, a, b, y : Code) ->
  Steps (App Proofs.rootRead
          (Proofs.quote (QApp (QApp (QApp (QApp p q) a) b) y)))
        Proofs.nTag
rootReadL3Deep p q a b y =
  stepsTrans (rootBeta (QApp (QApp (QApp (QApp p q) a) b) y)) $
  stepsTrans (appLs (appLs (readKDeep (QApp p q) a b y))) $
  stepsTrans (nTagCase
    (App readS (quote (QApp (QApp (QApp (QApp p q) a) b) y))) jWrap) $
  readSL3Deep p q a b y
```

## References


1. Project secret3, ‘The First Eye’ and ‘The Second Eye and the Assembler’, working papers v0.1, August 11, 2026.
2. Project secret3, ‘The Reader’s Engine’, working paper v0.1, August 11, 2026, whose root fragment this movement internalizes.
3. Project secret3, research agenda v0.13, line C5.